BASIC NETWORK SNIFFER USING PYTHON AND SCAPY

INTRODUCTION

Network traffic analysis is an essential aspect of cybersecurity and network administration. Understanding how data travels across a network helps security professionals identify malicious activities, troubleshoot connectivity issues, and gain insight into network behavior. Packet sniffing is the process of capturing and inspecting data packets as they travel through a network. This project focuses on developing a basic network sniffer using Python and the Scapy library. The program captures live network packets and extracts useful information such as source and destination IP addresses, protocols, and packet payloads. Through this project, the fundamentals of network communication and protocol analysis are explored.

AIM AND OBJECTIVES

Aim

To develop a Python-based network packet sniffer capable of capturing and analyzing network traffic in real time.

Objectives

- To capture live network packets using Python
- To analyze packet structures and contents
- To understand how data flows through a network
- To identify common network protocols
- To display useful packet information such as IP addresses, protocols, and payloads

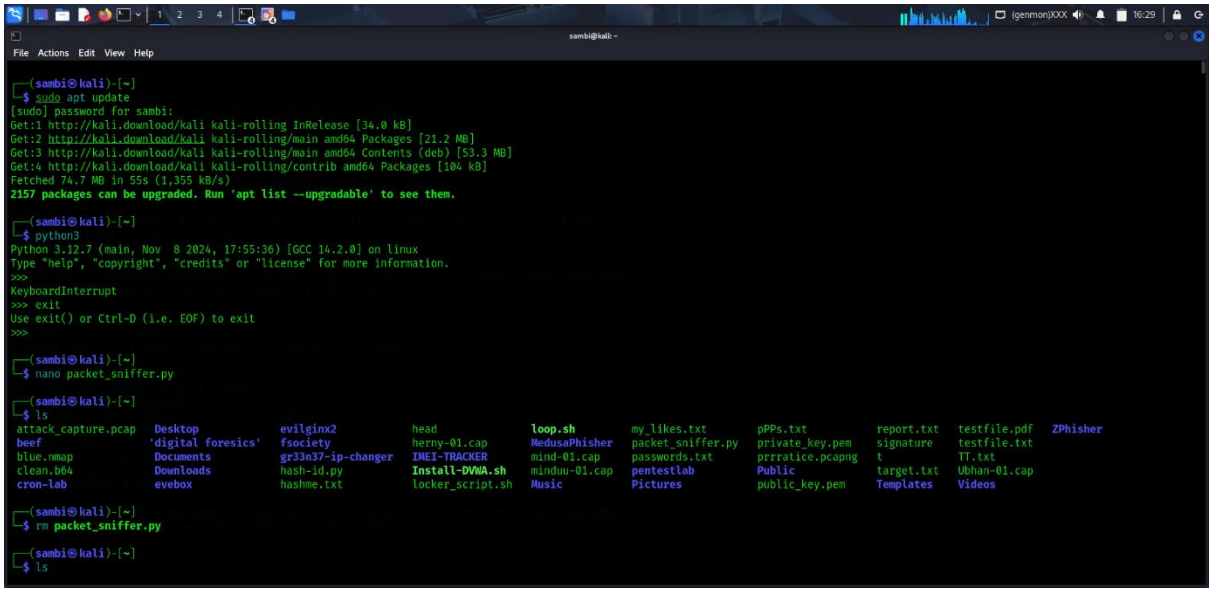
---

TOOLS AND ENVIRONMENT

The following tools and technologies were used:

- Python 3
- Scapy Library
- Kali Linux
- Terminal Environment

Scapy was selected because it provides powerful packet manipulation and packet capture capabilities suitable for network analysis tasks.



📌 Figure 1: *Successful installation of Scapy or python3 in terminal*

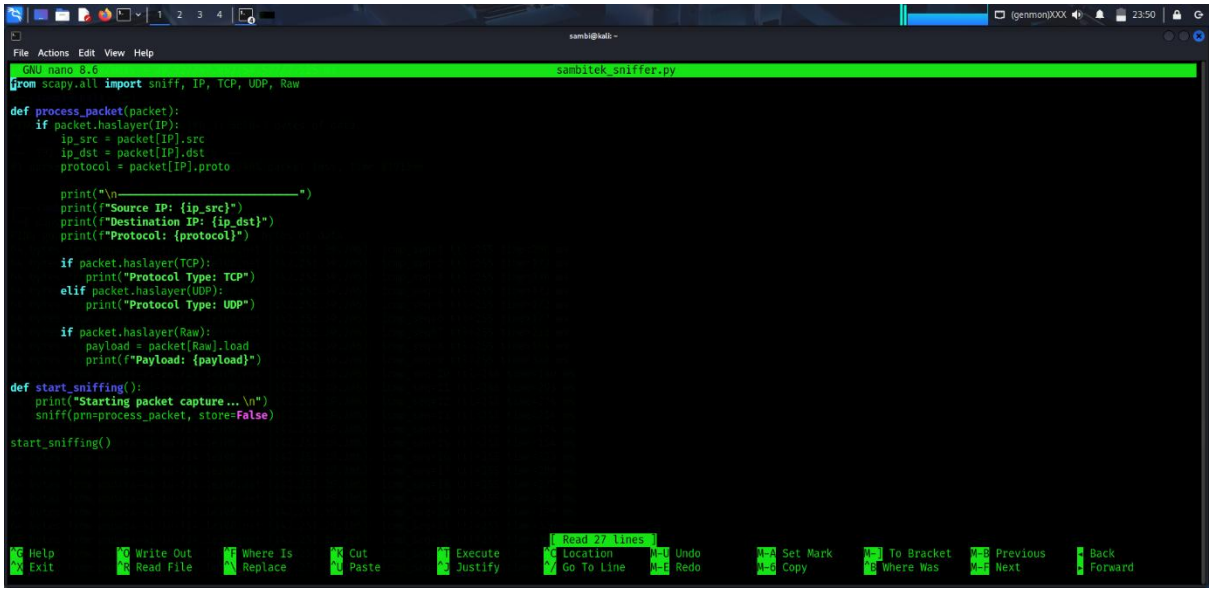
PROGRAM DEVELOPMENT

A Python script was developed using the Scapy library to capture network packets and extract relevant information.

The program performs the following functions:

- Captures packets from the network interface
- Identifies IP packets
- Displays source and destination IP addresses
- Detects TCP and UDP protocols
- Extracts and displays packet payloads when available

The core packet capture function continuously listens for network traffic and processes packets in real time.



📌 Figure 2*: Complete Python packet sniffer code*

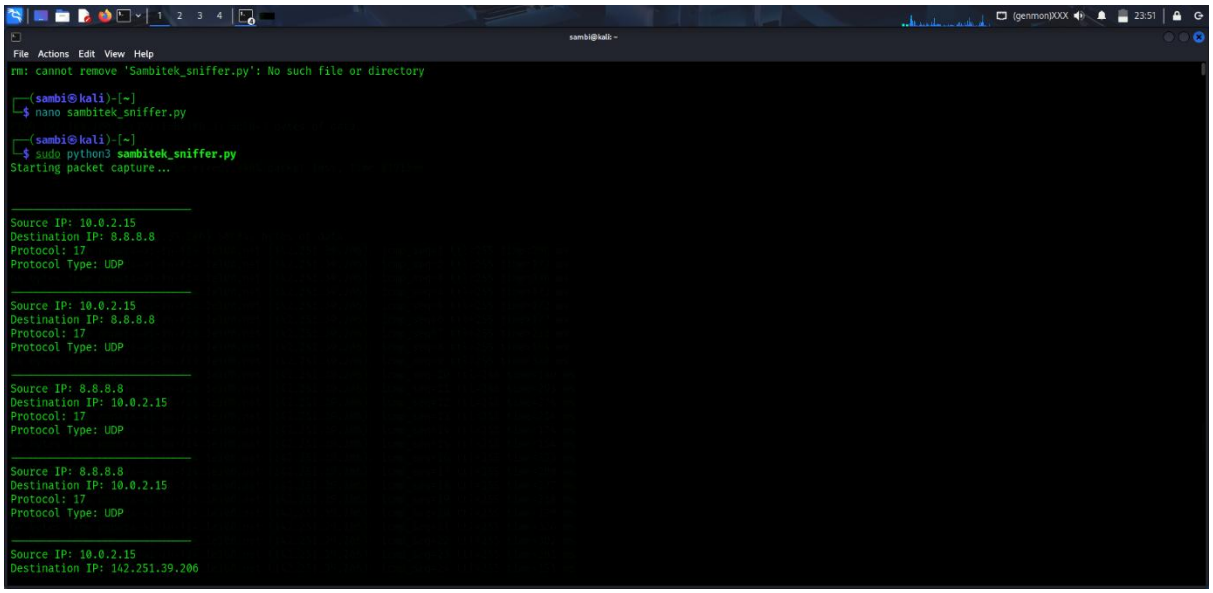
PACKET CAPTURE EXECUTION

The packet sniffer was executed with administrative privileges to allow access to network interfaces.

Command used:

sudo python3 sambitek_sniffer.py

Once launched, the program continuously monitored network traffic and displayed captured packet information on the terminal.



📌 *Figure 3: Packet sniffer running successfully in terminal*

TRAFFIC GENERATION

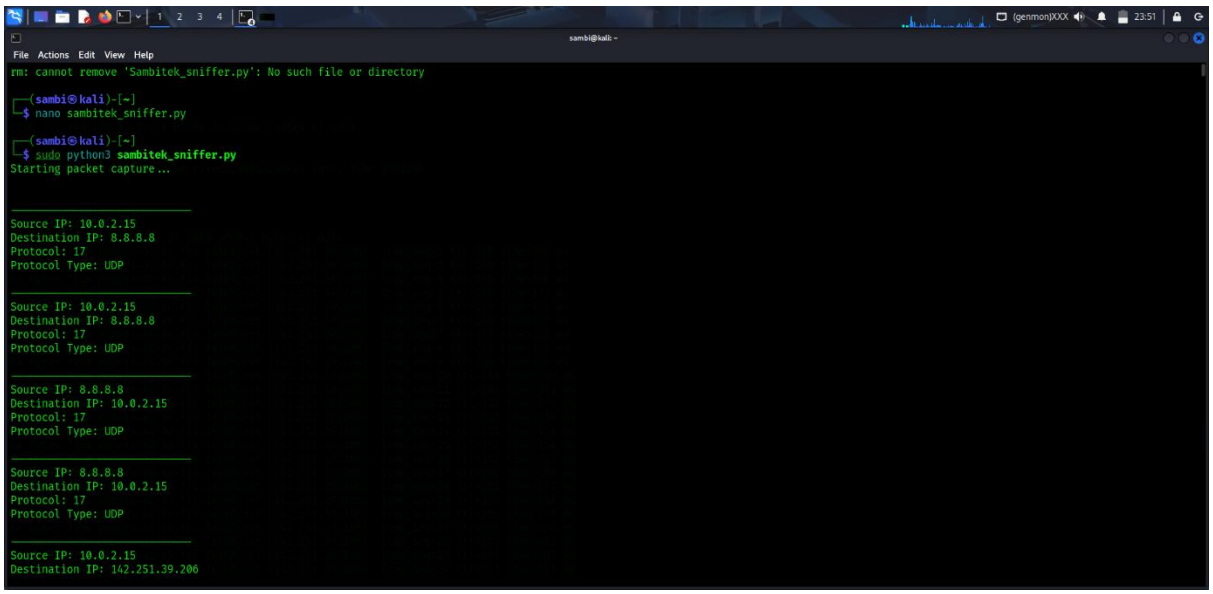
To generate network traffic for analysis, several activities were performed, including:

- Visiting websites through a web browser
- Sending ICMP requests using the ping command
- Performing network scans using Nmap

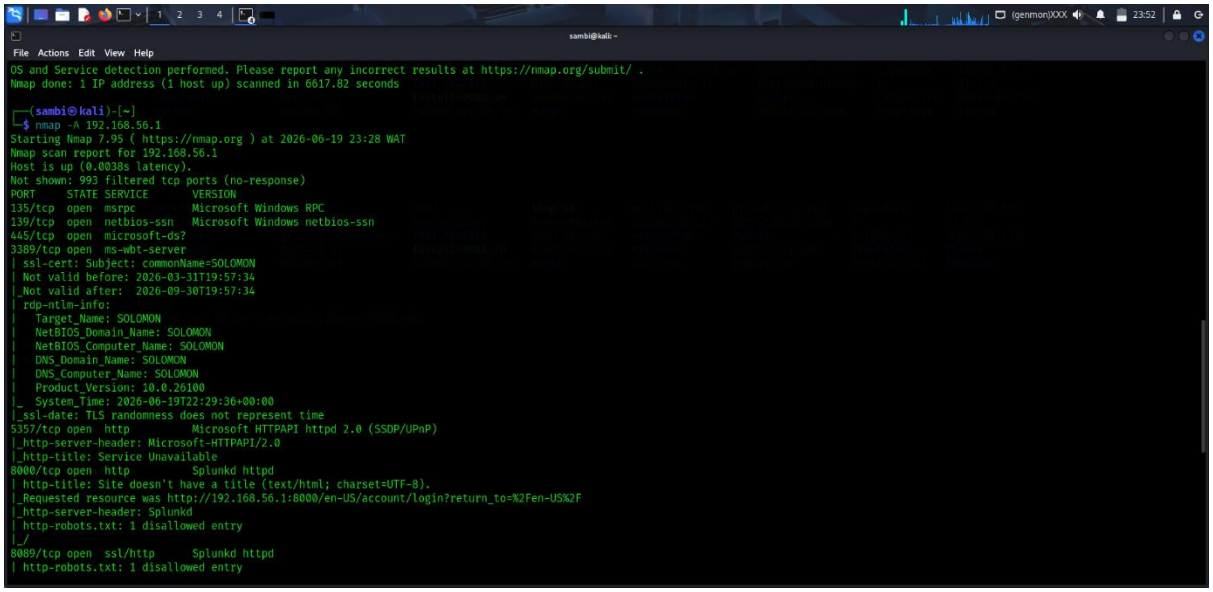
These activities generated packets that were captured and analyzed by the packet sniffer.

Example command used:

ping google.com



📌 *Figure 4 Traffic generation using ping command*



📌 *Figure 5: Nmap scan or web browsing activity used to generate traffic*

PACKET ANALYSIS

The captured packets revealed important network information, including:

Source and Destination IP Addresses

The program successfully identified the originating and receiving IP addresses of captured packets. This information helps track communication between devices on a network.

Protocol Identification

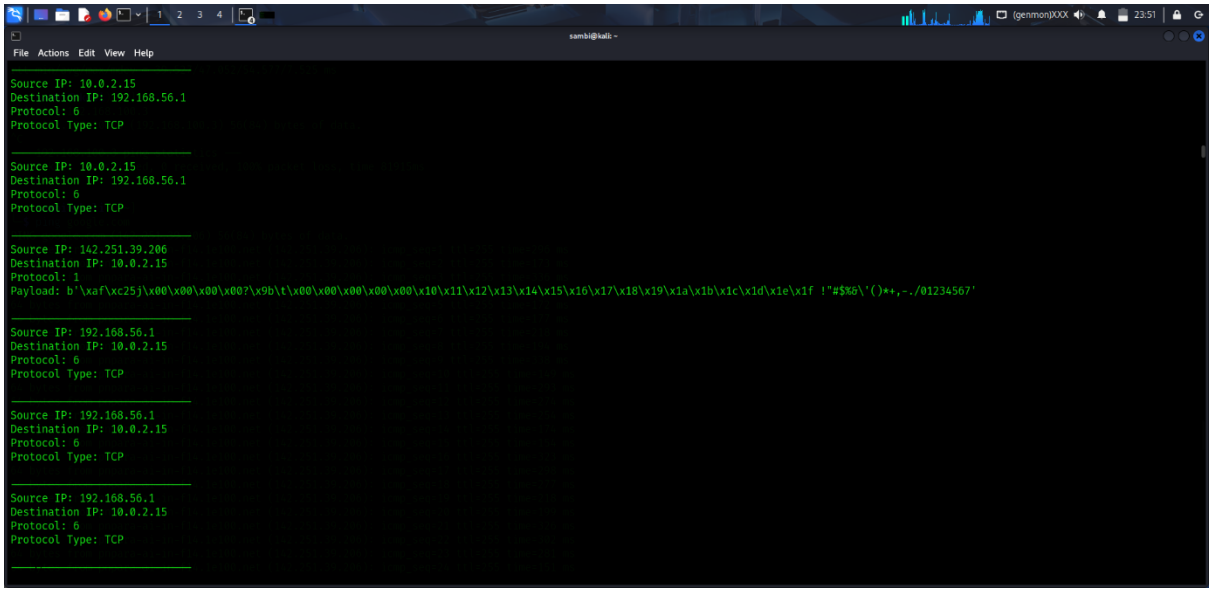
The packet sniffer detected common protocols such as:

- TCP (Transmission Control Protocol)
- UDP (User Datagram Protocol)

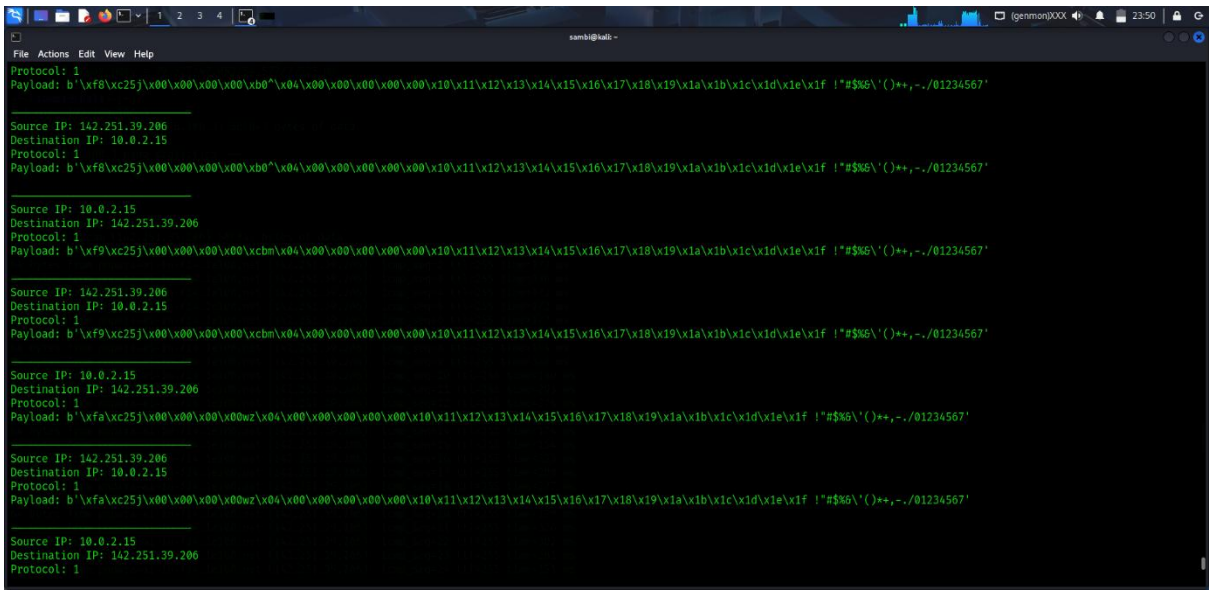
Protocol identification provides insight into the type of communication occurring on the network.

Payload Analysis

For packets containing data, the payload was displayed. This demonstrated how application data is transmitted across networks and how packet contents can be inspected.



📌 *Figure 6: Terminal output showing source IP, destination IP, and protocol information*



📌 Figure 7: *Packet output displaying payload data*

UNDERSTANDING NETWORK DATA FLOW

The project provided practical insight into how information flows across a network. When a user accesses a website or sends a request, data is divided into packets and transmitted between devices. Each packet contains addressing information and protocol details that enable successful communication.

By observing captured packets, it was possible to understand:

- How devices communicate using IP addresses
- How protocols facilitate data transmission

- How requests and responses travel between systems
- How packet payloads carry application data

FINDINGS

The following observations were made during the project:

- Network traffic can be captured and analyzed using Python and Scapy
- Source and destination IP addresses provide visibility into network communications
- TCP and UDP are among the most commonly observed protocols
- Packet payloads reveal the actual data being transmitted
- Packet sniffing provides valuable insight into network behavior and security monitoring

---

CONCLUSION

This project successfully demonstrated the development of a basic network sniffer using Python and Scapy. The application captured live network traffic and displayed essential packet information, including IP addresses, protocols, and payloads. The exercise provided practical experience in packet analysis and improved understanding of network communication processes. It also highlighted the importance of packet monitoring in cybersecurity, network troubleshooting, and traffic analysis. The project serves as a foundation for more advanced network monitoring and intrusion detection techniques.